Лабораторная работа № 3
Разработка REST API на Spring Boot

Проект **Library** • Java 21 • Spring Boot 4.1.2 • Maven

Пакеты: kg.course.library • kg.course.library.web • kg.course.library.config

Дата реализации: 23.10.2026

Подробный технический гайд по архитектуре, DTO, контроллерам, обработке ошибок, Swagger UI, тестам и подготовке к лабораторной № 4.

Оглавление

Placeholder for table of contents **0**

Глава 1. Общая информация

1.1. Название и цель работы

Лабораторная работа № 3 «Разработка REST API на Spring Boot». Цель — освоить создание веб-приложения на Spring Boot; изучить принципы REST и семантику методов и кодов HTTP; научиться разрабатывать REST-контроллеры, использовать DTO и валидацию; освоить централизованную обработку ошибок в формате Problem Details (RFC 9457); документировать API с помощью OpenAPI (Swagger UI).

1.2. Краткие условия задания

- Все эндпоинты имеют префикс /api, данные передаются в формате JSON.
- Работа выполняется на основе классов лабораторной работы № 2.
- Контроллеры, DTO и обработчик ошибок — в пакете kg.course.library.web.
- Конфигурация — в пакете kg.course.library.config.
- Проверка работоспособности — через Swagger UI и интеграционные тесты MockMvc.

1.3. Реализованные задачи

№	Задание	Результат
1	Создание проекта	pom.xml, перенос доменных классов, метод Library.removeBook
2	Конфигурация и данные	LibraryConfig, бины Library, Clock, ApplicationRunner, импорт CSV
3	DTO и валидация	records CreateBookRequest, CreateReaderRequest, IssueRequest и ответы
4	Книги	BookController: GET/POST/DELETE, поиск по названию
5	Читатели и выдачи	ReaderController, LoanController с возвратом и фильтром
6	Отчёты	ReportController: top, by-category, authors
7	Ошибки	ApiExceptionHandler, Problem Detail, свойство errors, безопасный 500
8	Документация	Swagger UI, /v3/api-docs, @Tag/@Operation
Доп	Интеграционные тесты	20 тестов MockMvc (@SpringBootTest, @AutoConfigureMockMvc, Clock.fixed)

1.4. Использованные технологии

Технология	Роль
Java 21	язык (классовые файлы версии 65)
Spring Boot 4.1.2	каркас, автоконфигурация, встроенный Tomcat
Spring Web (MVC)	REST-контроллеры, RestController
Spring Validation	аннотации Bean Validation (зависимость starter-validation)
springdoc-openapi 3.1.1	OpenAPI и Swagger UI
Jackson	сериализация JSON
JUnit 5 + MockMvc	интеграционные тесты (starter-webmvc-test)
Maven	сборка

1.5. Что перенесено из лабораторной № 2

Перенесены все классы домена kg.course.library кроме Main: Library, Book, Isbn, Reader, ReaderCategory, Loan, LibraryReports, BookCsvImporter, ReportWriter, ImportResult, ImportedBook, BookStat, LibraryException и производные исключения, а также вспомогательные классы (LibraryUtils, ReaderCard, Item, PaperBook, Magazine, Describable). Юнит-тесты LibraryTest и LibraryLab2Test перенесены без изменений. Бизнес-правила сохранились.

1.6. Что добавлено и изменено

- Новые пакеты: web (контроллеры, DTO, ошибки) и config.
- LibraryApplication — точка входа Spring Boot (@SpringBootApplication).
- Library.removeBook(Isbn): 404 если книги нет, 409 если есть активные выдачи.
- LibraryConfig — бины Library, Clock, LibraryReports, ApplicationRunner.
- Интеграционные тесты + TestClockConfig + TestBoomController.
- README, API_EXAMPLES.md, LAB3_COMPLETE_GUIDE.md/.pdf, LAB4_AGENT_HANDOFF.md.

Глава 2. Архитектура проекта

2.1. Структура каталогов и пакетов

```
library/ ├── pom.xml — зависимости, сборка ├── README.md — краткое описание проекта ├── books.csv — файл импорта (рабочий каталог) ├── docs/ — документация (MD и PDF) └── src/ ├── main/ │ ├── java/kg/course/library/ — домен + точка входа │ │ ├── config/LibraryConfig.java — бины и начальные данные │ │ ├── web/ — контроллеры, DTO, ошибки │ │ ├── LibraryApplication.java — @SpringBootApplication │ │ └── ... — доменные классы │ └── resources/application.properties — настройки └── test/java/kg/course/library/ — тесты
```

2.2. Назначение главных классов

Класс	Ответственность
Library	фасад бизнес-логики: каталог, копии, читатели, выдачи
Book, Isbn, Reader, Loan	value-объекты доменной модели (records)
ReaderCategory	enum с правилами категорий (дни лимита)
LibraryReports	отчёты и поиск по названию
BookCsvImporter	импорт books.csv в ImportResult
BookController	REST для книг
ReaderController	REST для читателей
LoanController	REST для выдач
ReportController	REST для отчётов
ApiExceptionHandler	централизованная обработка ошибок
DTO (records)	граница между API и доменом

2.3. Взаимодействие домен — контроллеры — DTO

```
[Клиент] --JSON--> [Контроллер web] --@Valid--> [Request DTO] | (внедрены Library, LibraryReports, Clock) v [Домен kg.course.library] (бизнес-правила, исключения) | v [Response DTO via from(...)] (не доменные объекты!) | v [Клиент] <--JSON/статус-- [Контроллер]
```

Контроллеры не содержат бизнес-логику: только преобразование JSON в DTO, вызов домена и формирование ResponseEntity (статус, заголовки).

2.4. Схема обработки запроса

```
HTTP-запрос --> DispatcherServlet --> HandlerMapping --> метод контроллера |-- @RequestBody + @Valid (валидация, иначе MethodArgumentNotValidException) |-- внедрённые Library / LibraryReports / Clock --> домен --> DTO --> ResponseEntity --> ответ JSON
```

2.5. Схема обработки исключений

```
Исключение (домен / валидация / прочее) | @RestControllerAdvice v ApiExceptionHandler (extends ResponseEntityExceptionHandler) | v ProblemDetail (RFC 9457) --> Content-Type: application/problem+json
```

Глава 3. Настройка окружения

3.1. Фрагмент pom.xml

```
org.springframework.boot spring-boot-starter-parent 4.1.2 kg.course library 1.0-SNAPSHOT 21 spring-boot-starter-web spring-boot-starter-validation org.springdoc:springdoc-openapi-starter-webmvc-ui:3.1.1 spring-boot-starter-webmvc-test (scope test)
```

3.2. Назначение зависимостей

Зависимость	Назначение
spring-boot-starter-web	Spring MVC, встроенный Tomcat, Jackson
spring-boot-starter-validation	Hibernate Validator (Bean Validation)
springdoc-openapi-starter-webmvc-ui:3.1.1	генерация OpenAPI и Swagger UI
spring-boot-starter-webmvc-test	JUnit 5, MockMvc, AssertJ (test)

3.3. application.properties

```
spring.application.name=library library.import-file=books.csv
```

Путь library.import-file внедряется через @Value и используется при импорте книг по запуске приложения.

3.4. Команды сборки, запуска, тестирования

```
cd library export JAVA_HOME=/home/atambek/.jdks/ms-21.0.12.1 # если JDK 21 не стоит по умолчанию mvn clean verify # полная сборка + тесты mvn spring-boot:run # запуск на :8080 mvn test -Dtest=LibraryApiIntegrationTest # только интеграционный тест
```

3.5. Типичные проблемы

Проблема	Решение
maven-jar-plugin не может скачать zstd-jni (нестабильная сеть)	докачать jar вручную в ~/.m2 (см. handoff) или повторять сборку
Spring Boot 4.1 vs 3.x — разные API тестов	пакет webmvc-тестов: org.springframework.boot.webmvc.test.autoconfigure
Приложение не стартует: UnsupportedClassVersionError	запускать через JDK 21, а не системный java 17

Глава 4. Доменная модель из лабораторной № 2

4.1. Основные классы

Класс	Описание
Library	каталог, копии, читатели, выдачи; операции issue/return/remove
Book	record (Isbn isbn, String title, List authors, int year) с валидацией
Isbn	record; нормализует и проверяет ISBN-10/ISBN-13
Reader	record (cardNumber, fullName, category)
ReaderCategory	enum: STUDENT(14,3), TEACHER(30,10), GUEST(7,1)
Loan	record; isActive(), isOverdue(today), returned(date)
LibraryReports	searchByTitle, overdueLoans, loansByCategory, allAuthors, topBooks

4.2. Ключевые бизнес-правила

- STUDENT — 14 дней, до 3 книг; TEACHER — 30 дней, до 10; GUEST — 7 дней, до 1.
- issueBook проверяет: читатель существует → нет просрочек → лимит категории → есть свободный экземпляр.
- returnBook: возврат по номеру выдачи; повторный возврат — IllegalStateException.
- Isbn убирает дефисы и пробелы: 978-5-4461-0914-9 → 9785446109149.
- removeBook (новое в лабе 3): 404 если нет, 409 если есть активные выдачи.

4.3. Как REST использует домен

Контроллеры вызывают методы Library и LibraryReports, но возвращают DTO. Исключения домена перехватываются ApiExceptionHandler и превращаются в Problem Detail.

4.4. Сохранённые / изменённые / дополненные классы

Класс	Статус
Library	дополнен: removeBook(Isbn)
Book, Isbn, Reader, ReaderCategory, Loan, LibraryReports, импортёры, исключения	сохранены без изменений
Main (лаба 2)	исключён из проекта (не REST-компонент)

Глава 5. DTO и валидация

5.1. Зачем нужны DTO

- Скрывают внутреннюю структуру доменных объектов.
- Позволяют менять домен, не ломая контракты API.
- Являются местом валидации входных данных.
- Не раскрывают домен наружу (принцип изоляции API).

5.2. Request-записи и валидация

Запись	Поля	Ограничения
CreateBookRequest	isbn, title, authors, year, copies	@NotBlank(isbn, title); @NotEmpty(authors); @Min(1450, year); @Min(1, copies)
CreateReaderRequest	cardNumber, fullName, category	@Pattern(R-\\d{5}); @NotBlank(fullName); @NotNull(category)
IssueRequest	cardNumber, isbn	@NotBlank оба

5.3. Response-записи и статические from(...)

- BookResponse(isbn,title,authors,year,totalCopies,availableCopies).from(Book, Library)
- ReaderResponse(cardNumber,fullName,category).from(Reader)
- LoanResponse(id,isbn,cardNumber,issuedOn,dueOn,returnedOn,overdue).from(Loan, LocalDate)

5.4. Примеры JSON

```
// корректный CreateBookRequest { "isbn": "978-5-4461-0914-9", "title": "Java. Эффективное программирование", "authors": ["Джошуа Блох"], "year": 2019, "copies": 2 } // некорректный (пустое название) { "isbn": "978-5-4461-0960-6", "title": " ", "authors": ["Роберт Мартин"], "year": 2019, "copies": 1 }
```

5.5. Как ошибка валидации превращается в HTTP-ответ

1) @Valid @RequestBody запускает Bean Validation. 2) При нарушении Spring бросает MethodArgumentNotValidException. 3) ApiExceptionHandler#handleMethodArgumentNotValid возвращает 400 ProblemDetail с полем errors — списком объектов {field, message}.

Глава 6. Конфигурация и начальные данные

6.1. LibraryConfig (@Configuration)

```
@Bean Library library() { return new Library(); } @Bean Clock clock() { return Clock.systemDefaultZone(); } @Bean LibraryReports libraryReports(Library library) { return new LibraryReports(library); } @Bean ApplicationRunner startupDataLoader(Library library, @Value("${library.import-file}") String importFile) { ... }
```

6.2. Бины Library и Clock

- library() — единственный экземпляр библиотеки.
- clock() — системные часы; LocalDate.now(clock) используется в контроллерах.

- libraryReports(...) — переиспользуемый компонент отчётов.
- startupDataLoader(...) — ApplicationRunner: импорт книг и регистрация читателей.

6.3. Импорт CSV и формат файла

```
Формат (разделитель «;», первая строка — заголовок): isbn;title;authors;year;copies
978-5-4461-0914-9;Java. Эффективное программирование;Джошуа Блох;2019;2
978-5-17-090630-2;Война и мир;Л. Н. Толстой;2015;1
```

6.4. Поведение при отсутствии файла

Если файл импорта отсутствует, при запуске выводится предупреждение в журнал и приложение стартует с пустым каталогом. Ошибки отдельных строк не останавливают импорт остальных корректных строк (каждая ошибка журналируется).

6.5. Регистрация трёх читателей

```
library.registerReader(new Reader("R-00001", "Нуртилек Калыгулов",
ReaderCategory.STUDENT)); library.registerReader(new Reader("R-00002", "Алексей
Смирнов", ReaderCategory.GUEST)); library.registerReader(new Reader("R-00003", "Иван
Петров", ReaderCategory.TEACHER));
```

6.6. Как изменить тестовую дату

В тестовой конфигурации TestClockConfig поле FIXED_DATE = LocalDate.of(2026, 3, 20). Бин назван fixedClock и помечен @Primary, поэтому внедряется по типу Clock вместо производственного. В контроллерах дата берётся через LocalDate.now(clock).

Глава 7. Все REST-контроллеры

7.1. BookController — /api/books

HTTP	Путь	Действие	Успех	Ошибки
GET	/api/books	список (с ?q= — поиск по названию)	200	—
GET	/api/books/{isbn}	книга по ISBN	200	404
POST	/api/books	добавление книги	201 + Location	400, 409
DELETE	/api/books/{isbn}	удаление	204	404, 409

Внедрение через конструктор: Library, LibraryReports. Повторное создание с тем же ISBN → IllegalStateException → 409. Удаление книги с активными выдачами → IllegalStateException → 409.

7.2. ReaderController — /api/readers

HTTP	Путь	Действие	Успех	Ошибки
GET	/api/readers	список читателей	200	—
GET	/api/readers/{cardNumber}	читатель по билету	200	404
POST	/api/readers	регистрация	201 + Location	400, 409
GET	/api/readers/{cardNumber}/loans	активные выдачи	200	404

Внедрение: Library, Clock. Повторная регистрация с тем же номером билета → IllegalStateException → 409.

7.3. LoanController — /api/loans

HTTP	Путь	Действие	Успех	Ошибки
GET	/api/loans	все выдачи (?overdue=true — просроченные)	200	—
GET	/api/loans/{id}	выдача по номеру	200	404
POST	/api/loans	выдача книги читателю	201 + Location	400, 404, 409
POST	/api/loans/{id}/return	возврат книги	200	404, 409

Внедрение: Library, LibraryReports, Clock. Даты выдачи/возврата — LocalDate.now(clock).

7.4. ReportController — /api/reports

HTTP	Путь	Действие	Успех	Ошибки
GET	/api/reports/top?n=5	N самых популярных книг	200	400 (n<1)
GET	/api/reports/by-category	выдачи по категориям	200	—
GET	/api/reports/authors	все авторы (алфавитный порядок)	200	—

Внедрение: LibraryReports. Расчёты не дублируются в контроллере — делегируются доменному LibraryReports. При n < 1 контроллер бросает IllegalArgumentException → 400.

Глава 8. HTTP и REST

8.1. Методы HTTP и идемпотентность

Метод	Назначение	Идемпотентен
GET	чтение ресурса	да
POST	создание / действие	нет
PUT	полная замена ресурса	да
PATCH	частичное изменение	нет
DELETE	удаление ресурса	да

8.2. @PathVariable, @RequestParam, @RequestBody

Аннотация	Откуда данные	Пример в этом проекте
@PathVariable	фрагмент URL	/api/books/{isbn}
@RequestParam	query-параметр	?q=, ?overdue=true, ?n=
@RequestBody	тело запроса (JSON)	CreateBookRequest и др.

8.3. Назначение ResponseEntity

ResponseEntity позволяет вернуть статус, заголовки (в том числе Location) и тело ответа. В проекте используется при создании ресурсов: ResponseEntity.created(URI.create(...)).body(dto).

8.4. Коды состояния, используемые в проекте

Код	Значение	Где встречается
200 OK	успех	GET, POST /return
201 Created	ресурс создан	POST /api/books, /api/readers, /api/loans
204 No Content	успех без тела	DELETE /api/books/{isbn}
400 Bad Request	неверные входные	валидация, неверный ISBN
404 Not Found	ресурс не найден	getBook/getReader/findLoan
409 Conflict	конфликт состояния	дубли, лимиты, активные выдачи
500 Internal Server Error	внутренняя ошибка	необработанные исключения

8.5. Почему важен заголовок Location

Location сообщает клиенту URL только что созданного ресурса (например, /api/books/9785446109149). Это позволяет клиенту обратиться к ресурсу сразу после создания и соответствует стандарту REST.

Глава 9. Обработка ошибок

9.1. ApiExceptionHandler и @RestControllerAdvice

ApiExceptionHandler — это @RestControllerAdvice, наследующий ResponseEntityExceptionHandler. Все исключения контроллеров обрабатываются централизованно и превращаются в ProblemDetail (RFC 9457) с Content-Type application/problem+json.

9.2. Соответствие исключений и HTTP-кодов

Исключение	HTTP-код
NotFoundException	404
BookUnavailableException	409
LimitExceededException	409
OverdueException	409
IllegalStateException	409
IllegalArgumentException	400
MethodArgumentNotValidException	400 (с полем errors)
прочие Exception	500 (без деталей, логируется)

9.3. Формат ProblemDetail

```
{ "type": "about:blank", "title": "Not Found", "status": 404, "detail": "Книга с ISBN 9780000000000 не найдена в каталоге", "instance": "/api/books/978-0-00-000000-0" }
```

Для ошибок валидации добавляется свойство errors — список объектов {field, message}:

```
{ "status": 400, "title": "Bad Request", "detail": "Переданные данные не прошли проверку", "errors": [ { "field": "title", "message": "Название книги обязательно" } ] }
```

9.4. Безопасный ответ 500

При необработанном исключении клиент получает общее сообщение «Внутренняя ошибка сервера» без текста исключения и без стектрейса. Полная диагностика записывается в журнал (log.error("Необработанное исключение", ex)). Это предотвращает утечку внутренней информации.

9.5. Как добавить обработку нового типа ошибки

```
// 1) класс исключения в kg.course.library public class MyException extends LibraryException { ... } // 2) обработчик в ApiExceptionHandler @ExceptionHandler(MyException.class) public ResponseEntity handleMy(MyException ex, WebRequest request) { return toResponse(ex, HttpStatus.BAD_REQUEST, ex.getMessage(), request); }
```

Глава 10. Swagger и OpenAPI

10.1. Назначение OpenAPI

OpenAPI — машинно-читаемое описание API (JSON). Из него Swagger UI строит интерактивную документацию, где можно выполнять запросы напрямую.

10.2. Доступные адреса

Адрес	Что это
http://localhost:8080/swagger-ui.html	Swagger UI (интерактивная документация)
http://localhost:8080/v3/api-docs	OpenAPI JSON (openapi 3.1.0)
http://localhost:8080/api/...	REST API приложения

10.3. Аннотации в коде

```
@RestController @RequestMapping("/api/books") @Tag(name = "Книги", description = "Управление каталогом книг") public class BookController { @GetMapping @Operation(summary = "Список книг", description = "...") public List findAll(...) { ... } }
```

10.4. Пошаговый сценарий проверки (соответствует заданию 8)

- POST /api/books — добавить книгу → 201 Created + Location.
- POST /api/readers — зарегистрировать читателя → 201 Created + Location.
- POST /api/loans — выдать книгу читателю → 201 Created + Location.
- POST /api/loans (ещё раз) — превышение лимита → 409 Conflict.

- POST /api/loans/{id}/return — вернуть книгу → 200 OK.
- GET /api/reports/top?n=5 — получить отчёт о популярных книгах → 200 OK.

10.5. Проверка кодов через curl

```
curl -s -i -X POST http://localhost:8080/api/books -H "Content-Type: application/json" -d '{"isbn":"978-5-4461-0960-6","title":"Чистый код","authors":["Роберт Мартин"],"year":2019,"copies":2}' # -> HTTP/1.1 409 (книга уже импортирована из books.csv) или 201 curl -s -i -X POST http://localhost:8080/api/readers -H "Content-Type: application/json" -d '{"cardNumber":"R-10001","fullName":"Тест","category":"GUEST"}' # -> HTTP/1.1 201, Location: /api/readers/R-10001
```

Глава 11. Интеграционные тесты

11.1. Что это такое

Интеграционные тесты запускают полный контекст Spring (@SpringBootTest) и отправляют HTTP-запросы через MockMvc без реального сервера. Так проверяется взаимодействие контроллеров, домена, валидации и обработки ошибок.

11.2. Ключевые аннотации

Аннотация	Назначение
@SpringBootTest	загружает полный ApplicationContext
@AutoConfigureMockMvc	создаёт бин MockMvc
@Import(TestClockConfig.class)	подключает фиксированные часы
@DirtiesContext(AFTER_EACH_TEST_METHOD)	изоляция синглтона Library между тестами

11.3. Как фиксируется время (Clock.fixed)

```
@TestConfiguration public class TestClockConfig { public static final LocalDate FIXED_DATE = LocalDate.of(2026, 3, 20); @Bean @Primary public Clock fixedClock() { return Clock.fixed(FIXED_DATE.atStartOfDay(ZoneId.systemDefault()).toInstant(), ZoneId.systemDefault()); } }
```

Бин назван fixedClock (не clock), чтобы не конфликтовать с производственным бином при запрещённом в Boot 4.1 переопределении бинов. @Primary заставляет внедрять по типу Clock именно этот бин.

11.4. Обзор покрываемых сценариев (20 тестов)

Тест	Проверяет
createBook_returns201AndLocation	201 + Location для POST /api/books
createBook_duplicate_returns409	409 при повторе ISBN
createBook_blankTitle_returns400WithTitleInErrors	400 и поле title в errors
createBook_invalidIsbn_returns400	400 при неверном ISBN
getBook_byIsbn_returns200 / _notFound_returns404ProblemJson	200 / 404 problem+json
searchBooks_byTitle_returnsMatching	поиск по ?q=
deleteBook_*	404 для отсутствующей; 409 для книги с выдачами
registerReader_*	201 + Location; 409 дубль; 400 неверный формат
issueAndReturnBook_success	201 выдача → 200 возврат
issueBook_overLimit_returns409	превышение лимита категории
loans_overdueFilter_returnsOnlyOverdueOnFixedDate	фильтр ?overdue=true
reports_top_*	граничные значения n (400 при n<1)
internalError_returns500WithoutExceptionDetails	безопасный 500 без деталей

11.5. Запуск тестов и интерпретация ошибок

```
mvn test -Dtest=LibraryApiIntegrationTest mvn test mvn clean verify
```

Отчёты Surefire лежат в target/surefire-reports/*.txt. Зелёная сборка: «Tests run: 33, Failures: 0, Errors: 0».

Глава 12. Подробный разбор исходного кода

12.1. Ключевые файлы и взаимосвязи

Файл	Назначение
Library.java	ядро бизнес-логики (+ removeBook)
book/web/BookController.java	REST книг
book/web/*.java (DTO)	records запросов/ответов
book/web/ApiExceptionHandler.java	Problem Detail
book/config/LibraryConfig.java	бины и стартовая загрузка

12.2. Обработка запроса POST /api/loans

```
// LoanController.issue Loan loan = library.issueBook(request.cardNumber(), new Isbn(request.isbn()), LocalDate.now(clock)); LoanResponse body = LoanResponse.from(loan, today); return ResponseEntity.created(URI.create("/api/loans/" + loan.id())).body(body);
```

Шаги: 1) валидация IssueRequest; 2) new Isbn нормализует ISBN; 3) Library.issueBook проверяет читателя, просрочку, лимит и доступность; 4) LoanResponse конвертирует домен в DTO; 5) 201 + Location.

12.3. Обработка 404

```
// Library.getBook бросает throw new NotFoundException("Книга с ISBN " + isbn.value() + "
не найдена в каталоге"); // ApiExceptionHandler
@ExceptionHandler(NotFoundException.class) public ResponseEntity
handleNotFound(NotFoundException ex, WebRequest request) { return toResponse(ex,
HttpStatus.NOT_FOUND, ex.getMessage(), request); }
```

12.4. Почему from(...) — важное архитектурное решение

Book не содержит полей totalCopies/availableCopies; эти значения вычисляет Library. DTO решает, что показывать клиенту, и не допускает утечки внутреннего представления домена.

Глава 13. Контрольные вопросы

1. Что такое REST и какие требования предъявляются к REST API?

REST — архитектурный стиль, основанный на ресурсах с URL, HTTP-методах и кодах статуса. Требования: клиент-сервер, отсутствие состояния, кеширование, единый интерфейс, слои, код по требованию (опционально). В проекте контроллеры представляют ресурсы, методы — операции, ответы — JSON.

2. Чем отличаются GET, POST, PUT, PATCH и DELETE? Какие методы идемпотентны?

GET — чтение, POST — создание/действие, PUT — полная замена, PATCH — частичное изменение, DELETE — удаление. Идемпотентны GET, PUT, DELETE (повторный вызов даёт тот же результат). POST и PATCH идемпотентными не являются.

3. Что означают коды 200, 201, 204, 400, 404, 409 и 500?

200 - успех; 201 - ресурс создан; 204 - успех без тела; 400 - неверные входные данные; 404 - ресурс не найден; 409 - конфликт состояния; 500 - внутренняя ошибка. В проекте: 201 при создании книг/читателей/выдач, 204 при удалении, 409 при дублях и лимитах, 404 при отсутствии ресурса.

4. Чем @RestController отличается от @Controller?

@RestController = @Controller + @ResponseBody: возвращаемые значения метода сразу сериализуются в JSON. @Controller при этом требует явной аннотации для тел ответов и обычно используется с представлениями.

5. Чем @PathVariable отличается от @RequestParam и @RequestBody?

@PathVariable - фрагмент пути URL; @RequestParam - query-параметр; @RequestBody - тело запроса (JSON). Примеры: /api/books/{isbn}, ?q=, CreateBookRequest.

6. Для чего используются DTO?

DTO отделяют API от доменной модели: скрывают внутреннюю структуру, контролируют сериализацию и валидацию, не позволяют «протекать» доменными объектами наружу.

7. Как работает @Valid и где обрабатывается ошибка валидации?

@Valid @RequestBody включает Bean Validation для DTO. При нарушении Spring бросает MethodArgumentNotValidException, которую обрабатывает ApiExceptionHandler#handleMethodArgumentNotValid, возвращая 400 ProblemDetail с полем errors (field/message).

8. Что такое внедрение зависимостей и почему предпочтителен конструктор?

DI — передача зависимостей объекту контейнером. Конструктор предпочтителен: зависимости явны и обязаны существовать, поля могут быть final, объект прост в тестировании (не требует рефлексии).

9. Для чего нужны @RestControllerAdvice и Problem Details?

@RestControllerAdvice централизует обработку исключений всех контроллеров. Problem Details (RFC 9457) даёт единый машиночитаемый формат ошибок (type,title,status,detail,instance) и Content-Type application/problem+json.

10. Для чего используется Clock вместо прямого вызова LocalDate.now()?

Clock — абстракция источника времени, внедряемая как бин. В тестах его заменяют на Clock.fixed(...), что делает «сегодня» предсказуемой датой и позволяет тестировать сценарии без изменения системного времени.

Глава 14. Подготовка к лабораторной работе № 4

Полный передаточный документ — файл LAB4_AGENT_HANDOFF.md. Ниже — краткая выжимка.

14.1. Текущее состояние

- Реализованы все задания 1-8 лабораторной № 3.
- 33 теста зелёные: 20 интеграционных + 8 + 5 юнит.
- Собрано target/library-1.0-SNAPSHOT.jar (27 МБ).

14.2. Технологический стек

Компонент	Версия
Java	21 (JDK /home/atambek/.jdks/ms-21.0.12.1)
Spring Boot	4.1.2
Spring Framework	7.0.x
springdoc	3.1.1
Spring-тесты	spring-boot-starter-webmvc-test (test scope)

14.3. Что нельзя сломать

- Бизнес-логика живёт только в домене; контроллеры — только HTTP.
- Контроллеры получают зависимости через конструктор.
- Доменные объекты не возвращаются из API напрямую — только DTO.
- Дата в контроллерах — только через Clock (LocalDate.now(clock)).
- Ошибки обрабатываются централизованно в ApiExceptionHandler.

14.4. Как безопасно добавлять функциональность

- Новый эндпоинт: метод в контроллере + DTO + (при нужде) метод в домене.
- Новое правило: класс исключения + обработчик в ApiExceptionHandler.
- Валидация: аннотации Bean Validation на request DTO.
- Тесты: MockMvc + фиксированный Clock; не зависеть от реальной даты.
- Не менять контракты существующих эндпоинтов и не реализовывать неизвестную лабу № 4 заранее (нет методички).

Глава 15. Итоги и дальнейшее развитие

15.1. Что реализовано

- Spring Boot-приложение Library с REST API на базе домена лабы № 2.
- Контроллеры книг, читателей, выдач и отчётов.
- DTO с валидацией и изоляцией домена.
- Централизованная обработка ошибок (RFC 9457).
- Swagger UI и OpenAPI.
- 20 интеграционных тестов MockMvc (задание «дополнительно»).

15.2. Что проверено

- mvn clean test: 33 теста без ошибок.
- mvn clean verify: jar собран (zstd-jni докачан при сбоях сети).
- Запуск приложения: Tomcat на 8080, started LibraryApplication.
- Swagger UI 200, /v3/api-docs 200 (openapi 3.1.0).
- REST-сценарий № 8: 201/201/201→409/200/200 — коды верны.
- Ошибки: 404 и 400 с problem+json, errors при валидации.

15.3. Ограничения

- Из-за нестабильной сети Maven Central требовалась докачка zstd-jni для jar-плагина.
- ISBN в ответах возвращается в нормализованной форме без дефисов (9785446109149).
- Классы Item/PaperBook/Magazine из лабы № 2 не задействованы в REST, но сохранены.
- Для запуска jar нужен JDK 21 (системный java — 17).

15.4. Как начать следующий этап

- Прочитать LAB4_AGENT_HANDOFF.md и LAB3_COMPLETE_GUIDE.md.
- Дождаться методички лабораторной № 4.
- Применять новые функции через домен + контроллер + тест.
- Обновлять README, API_EXAMPLES и данное руководство при изменениях.